

A Comparative Analysis of Dynamic Programming and Temporal-Difference Learning on Discrete and Continuous Control Problems

Shiyu Luo

Georgia Institute of Technology

Atlanta, Georgia 30332

Email: sluo95@gatech.edu

Abstract—This paper presents a rigorous empirical analysis of foundational reinforcement learning (RL) algorithms applied to two distinct Markov Decision Processes (MDPs): the discrete, stochastic Blackjack-v1 environment and the continuous, deterministic CartPole-v1 control problem. We implement and evaluate model-based dynamic programming methods, specifically Value Iteration (VI) and Policy Iteration (PI), and the model-free temporal-difference algorithm, SARSA. A key focus of this investigation is the challenge posed by continuous state spaces, which we address by developing a state discretization strategy for the CartPole environment and analyzing its impact on the trade-off between solution accuracy and computational complexity. The performance of these algorithms is systematically compared based on convergence rates, final policy quality, and reward maximization. To explore solutions beyond tabular methods, we implement a Deep Q-Network (DQN) for the CartPole problem. This allows for a direct comparison between tabular and function approximation approaches, highlighting the latter’s ability to generalize in continuous domains. The results provide a clear exposition of the strengths and weaknesses of each algorithm, offering insights into their practical application and theoretical underpinnings.

Index Terms—Reinforcement Learning, Markov Decision Process, Value Iteration, Policy Iteration, SARSA, Deep Q-Network, Function Approximation, Dynamic Programming.

I. INTRODUCTION

Reinforcement Learning (RL) provides a powerful framework for training agents to make optimal decisions in complex and uncertain environments. This is achieved by learning a policy, a mapping from states to actions, that maximizes a cumulative reward signal. The methods for finding such a policy can be broadly categorized into model-based and model-free approaches. Model-based methods, such as those from dynamic programming rooted in Bellman’s work [1], require a complete model of the environment’s dynamics, while model-free methods learn directly from interaction, making them more applicable to real-world scenarios where such models are unavailable.

This study investigates these paradigms by applying them to two canonical RL problems that embody different fundamental challenges. The Blackjack-v1 environment is an archetype for discrete, stochastic problems with sparse rewards, where an exact optimal solution is computable. In contrast, the CartPole-v1 environment is a classic continuous control problem, where

the infinite state space makes exact tabular solutions impossible and necessitates approximation techniques.

This paper investigates three key questions:

- 1) How do the computational and convergence properties of model-based VI and PI compare in a finite, discrete state space?
- 2) What are the performance limitations of a tabular model-free agent (SARSA) in a discretized continuous state space, and how do choices regarding exploration and discretization granularity affect learning?
- 3) How effectively can deep function approximation with a DQN overcome the limitations of tabular methods in a continuous control task?

Through detailed analysis of the empirical results, this paper aims to provide a clear and deep understanding of the practical trade-offs and theoretical differences between these foundational RL algorithms.

II. METHODOLOGY

A. Markov Decision Process Environments

1) *Blackjack-v1*: This environment models the card game Blackjack. Its state is defined by a 3-tuple: the player’s current hand sum, the dealer’s visible card, and whether the player holds a “usable” ace. The stochasticity arises from the random drawing of cards from the deck. The sparse, terminal reward structure (+1 for a win, -1 for a loss) makes credit assignment challenging, as the agent only receives feedback at the end of an episode.

2) *CartPole-v1*: This is a classic physics-based control problem. The agent’s goal is to balance a pole on a cart by applying horizontal forces. The state is a 4-dimensional continuous vector. Although the environment’s physics are deterministic for a given action, the continuous nature of the state space makes it intractable for tabular methods, which would require an infinitely large table. This property makes it an ideal candidate for comparing discretization and function approximation techniques.

B. Algorithm Implementation

1) *Value and Policy Iteration*: These model-based algorithms were implemented for the Blackjack environment. They can be understood as applying Bellman operators to find the

optimal value function V^* . VI iteratively applies the Bellman optimality operator, as shown in (1), which is guaranteed to converge to V^* for a discount factor $\gamma < 1$.

$$V_{k+1}(s) \leftarrow \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')] \quad (1)$$

PI alternates between applying the Bellman expectation operator for a fixed policy (policy evaluation) and making the policy greedy with respect to the resulting value function (policy improvement).

2) *SARSA*: The on-policy temporal-difference algorithm SARSA was implemented for both environments. Being on-policy means it learns the value of the policy it is currently following, including its exploratory actions. This is evident in its update rule, shown in (2), which uses the action a' actually taken in the next state. This contrasts with off-policy methods like Q-Learning, which would use the greedy action $\max_{a'} Q(s', a')$ regardless of the policy being followed.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (2)$$

3) *State Discretization*: To apply tabular methods to CartPole, its continuous state space was converted into a finite grid. This process, also known as state aggregation, inevitably loses information and can lead to perceptual aliasing, where distinct continuous states are mapped to the same discrete representation. The number of bins is a critical hyperparameter that was chosen through experimentation, as shown in the analysis section. A uniform grid was used, though more advanced techniques like adaptive discretization could potentially yield better performance by allocating more bins to critical regions of the state space.

4) *Deep Q-Network (DQN)*: For the CartPole problem, a DQN was implemented. This approach uses a neural network as a function approximator to estimate the Q-function, $Q(s, a; \theta) \approx Q^*(s, a)$. To ensure stable training, two key techniques from the original DQN paper [2] were used:

- **Experience Replay**: This technique stores past transitions in a large replay buffer. During training, mini-batches are sampled randomly from this buffer. This breaks the strong temporal correlations in consecutive experiences, satisfying the I.I.D. data assumption of many optimization algorithms and leading to more stable learning.
- **Target Network**: A separate target network, with weights that are periodically copied from the main policy network, is used to generate the TD targets. This creates a stable, consistent target for the Q-value updates, preventing the instability that can arise from having the network "chase a moving target."

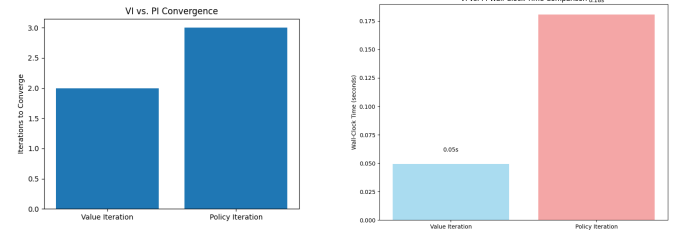
The network architecture consisted of an input layer, two hidden layers with 64 neurons each and ReLU activation, and an output layer with linear activation producing Q-values for each action. The Adam optimizer was used for training.

III. EXPERIMENTAL RESULTS AND ANALYSIS

A. Value Iteration vs. Policy Iteration

1) *Convergence Speed Analysis*: **Observation**: The empirical results in Fig. 1 show that Value Iteration converged in 2 iterations while Policy Iteration took 3. However, VI was substantially faster in wall-clock time (0.05s) compared to PI (0.18s).

Analysis and Reasoning: This result highlights the crucial difference in computational complexity per iteration. A single VI iteration is computationally lightweight, involving one sweep through the state space to perform the Bellman update. In contrast, a PI iteration contains a full policy evaluation step, which is itself an iterative algorithm that must sweep the state space repeatedly until the value function for the current policy converges. Therefore, even when PI takes fewer iterations on more complex problems by making larger "jumps" in policy space, the high cost of each iteration can make it slower overall. For a small state space like Blackjack, VI's simpler updates proved more efficient.



(a) Iterations to Converge. (b) Wall-Clock Time (seconds).

Fig. 1: VI vs. PI convergence comparison for Blackjack.

2) *Optimal Policy Analysis*: **Observation**: Both VI and PI converged to the identical optimal policy. The resulting policies, visualized for hands without and with a usable ace in Fig. 2 and Fig. 3 respectively, are distinct and reveal optimal strategy.

Analysis and Reasoning: The learned policies align perfectly with established Blackjack strategy, demonstrating that both algorithms successfully found the optimal solution. For "hard" hands without a usable ace (Fig. 2), the agent learns to be risk-averse, standing on any total of 17 or higher because the expected value of hitting is negative, as the high probability of busting outweighs the chance of improving the hand. For "soft" hands with a usable ace (Fig. 3), the agent is more aggressive. Because the ace can be valued at 1 or 11, the hand cannot bust on a single hit. The agent correctly learns that it can safely "hit" on totals like a soft 17 or 18 to improve its hand without risk, a nuance that demonstrates the power of these algorithms to find non-obvious optimal actions.

B. SARSA Performance Analysis

1) *Reward Maximization*: **Observation**: The SARSA agent for Blackjack converges to an average reward near -0.2 (Fig. 4), while the CartPole agent plateaus at a low average reward of 12-15 (Fig. 5).

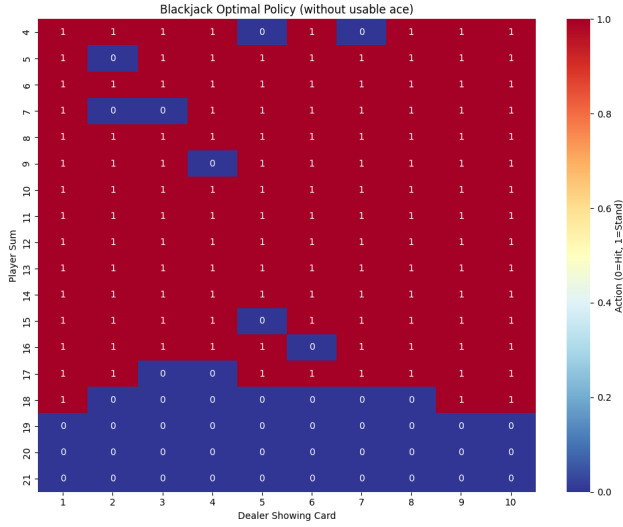


Fig. 2: Optimal Policy without Usable Ace (0=Hit, 1=Stand).

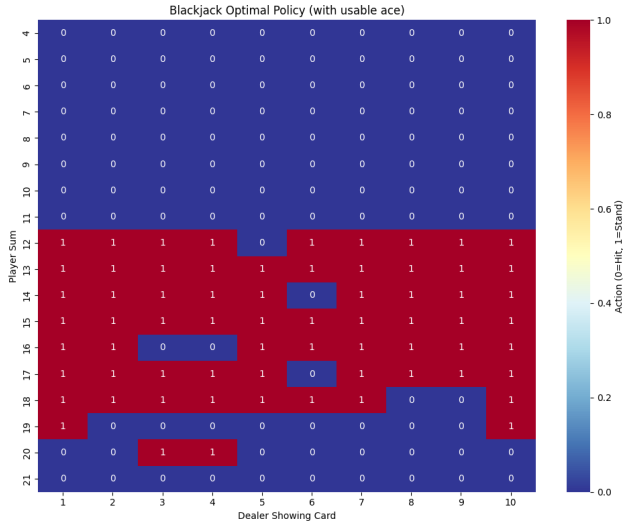


Fig. 3: Optimal Policy with Usable Ace (0=Hit, 1=Stand).

Analysis and Reasoning: For Blackjack, a negative reward is optimal due to the house edge. Converging to a small negative number indicates the agent has successfully learned the best possible strategy to minimize its losses over time. For CartPole, the low plateau demonstrates the limitations of the tabular approach. The agent learns a simple “wobble” strategy but cannot achieve long-term stability. This is a direct consequence of perceptual aliasing; the coarse discretization prevents the agent from distinguishing between a state where the pole is slowly drifting and one where it is about to fall, so it cannot learn the precise, corrective actions required.

2) *Exploration Strategy Analysis:* **Observation:** The decaying epsilon strategy dramatically outperforms the constant epsilon strategy (Fig. 6). The decaying strategy shows clear phases of learning, while the constant strategy stagnates.

Analysis and Reasoning: This result is a textbook demon-

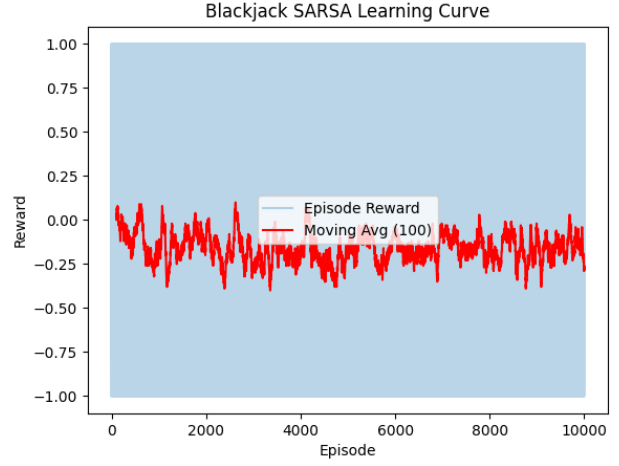


Fig. 4: Blackjack SARSA Learning Curve over 10,000 episodes.

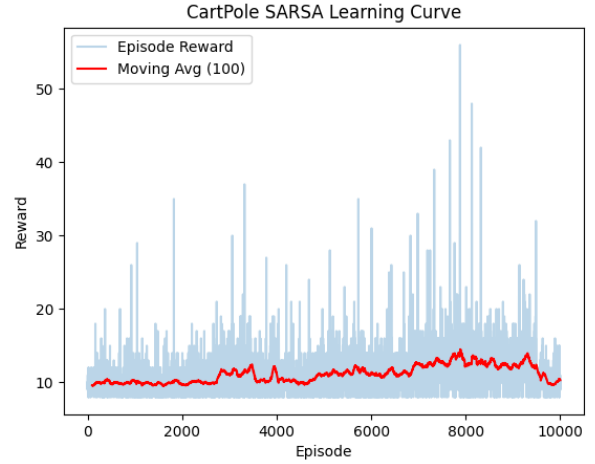


Fig. 5: CartPole SARSA Learning Curve over 10,000 episodes.

stration of the exploration-exploitation dilemma. A constant epsilon forces the agent into perpetual exploration, preventing the policy from ever becoming fully greedy and stable. It continuously discards what it has learned for a random action, crippling its maximum performance. A decaying epsilon provides a principled solution: it encourages high exploration early on to build a robust model of the environment (the initial slow growth phase), then gradually reduces exploration to exploit that learned knowledge (the steep learning phase), allowing the policy to converge to a much better solution (the final plateau).

C. Effect of Discretization on CartPole

Observation: Increasing the number of discretization bins improves the final average reward but also increases the computation time (Fig. 7). The performance gains show diminishing returns, while computation time grows non-linearly.

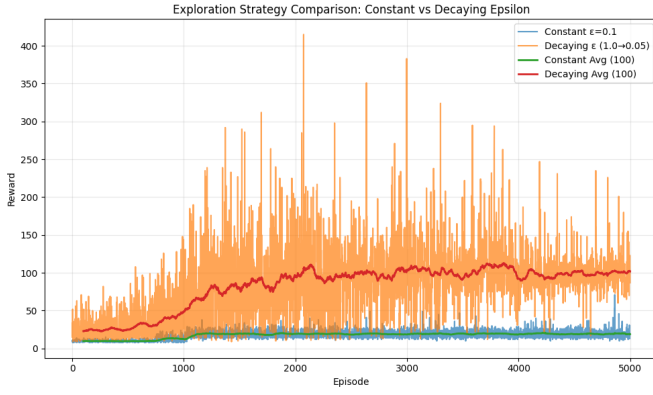


Fig. 6: Comparison of exploration strategies for SARSA.

Analysis and Reasoning: This demonstrates the fundamental trade-off of discretization. More bins lead to a higher-fidelity state representation, reducing information loss and allowing the agent to learn a more precise control policy. However, this comes at the cost of an exponential growth in the state space size (the curse of dimensionality), which requires more memory and significantly more training time to explore adequately. The diminishing returns on performance suggest that at a certain point, the tabular agent is unable to effectively leverage the finer state distinctions, while the computational cost continues to rise sharply.

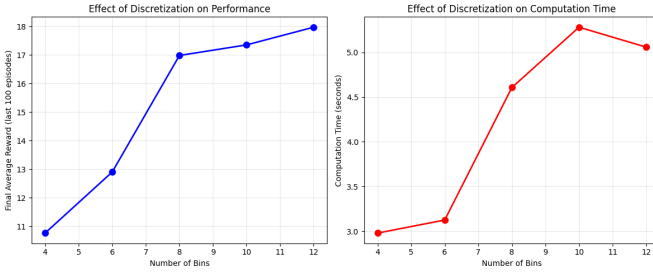


Fig. 7: Effect of discretization on performance and computation time.

IV. EXTRA CREDIT: DEEP Q-NETWORK ANALYSIS

A. DQN vs. Tabular Q-Learning Performance

Observation: The DQN agent’s performance (Fig. 8) far surpasses the tabular SARSA agent’s. The direct comparison in Fig. 9 shows the DQN achieving rewards near the maximum of 500, while tabular SARSA flatlines below 20.

Analysis and Reasoning: The performance gap is explained by *generalization*. The DQN’s neural network learns a smooth, continuous function that approximates the Q-values across the entire state space. It can interpolate and make intelligent decisions for states it has never seen before. Tabular methods cannot do this; they treat each discrete bin as an isolated state. This ability to generalize is what allows the DQN to learn the complex, non-linear relationships in the environment’s dynamics. The trained DQN agent exhibits a

sophisticated control strategy, making small, corrective nudges to keep the pole almost perfectly vertical—a behavior impossible for the tabular agent.

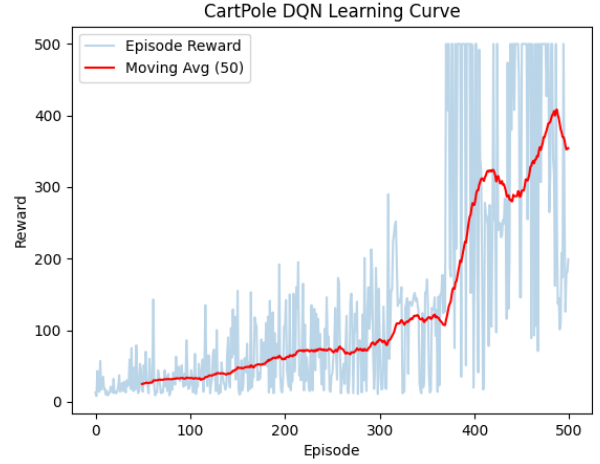


Fig. 8: CartPole DQN Learning Curve.

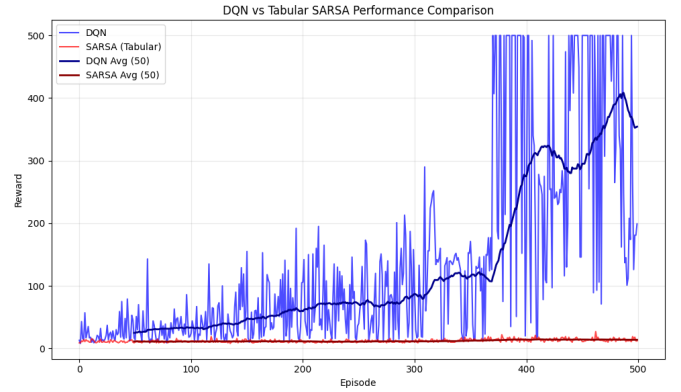


Fig. 9: DQN vs. Tabular SARSA Performance on CartPole.

V. CONCLUSION

A. Summary of Findings

This study provided a comparative analysis of foundational reinforcement learning algorithms. Our key findings are: 1) For discrete problems with known models, model-based methods like VI are extremely efficient. 2) For continuous state spaces, tabular methods are severely constrained by the curse of dimensionality, facing a direct trade-off between performance and computational cost. 3) Function approximation with DQNs effectively overcomes this limitation by learning to generalize, achieving far superior performance on continuous control tasks. 4) A proper exploration strategy that transitions from exploration to exploitation is critical for the success of model-free learning.

B. Limitations and Future Work

This study was limited to two specific environments and a basic implementation of each algorithm. The DQN, for example, could be improved with more advanced techniques from the "Rainbow" paper [4], such as Prioritized Experience Replay or Dueling Networks. Future work should involve applying these advanced algorithms to more complex, high-dimensional control problems from sources like the OpenAI Gym [3] to further probe their capabilities and limitations.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [3] G. Brockman et al., "OpenAI Gym," arXiv:1606.01540 [cs.LG], 2016.
- [4] M. Hessel et al., "Rainbow: Combining Improvements in Deep Reinforcement Learning," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.